



CodeSmile

Change Request

RIFERIMENTO	ChangeRequest, UseCase
VERSIONE	1.5
DESTINATARIO	Prof. Andrea De Lucia Dott. Giammaria Giordano



Laurea Magistrale in Software Engineering & IT Management –Università di Salerno
Corso di Ingegneria del Software: tecniche avanzate
Prof. Andrea De Lucia, Dott. Giammaria Giordano

Gestione Gruppo

MEMBRO	EMAIL	MATRICOLA
MARTINA DE LUCIA	m.delucia18@studenti.unisa.it	NF22500053
ALESSANDRA RAIA	a.raia7@studenti.unisa.it	NF22500054
CARLA STEFANILE	c.stefanile1@studenti.unisa.it	NF22500097



Version

DATA	VERSIONE	DESCRIZIONE	AUTORI
16/01/2026	0.1	Prima Stesura	MDL, AR, CS
16/01/2026	0.2	Introduzione	AR
16/01/2026	0.3	Prima stesura CR	AR
17/01/2026	0.4	Prima revisione	MDL, CS
17/01/2026	0.5	Seconda stesura CR	MDL
17/01/2026	0.6	Seconda Revisione	AR, CS
18/01/2026	0.7	Requisiti funzionali e non funzionali	MDL
18/01/2026	0.8	Terza Revisione + Introduzioni intermedie + Note	AR
19/01/2026	0.9	Quarta Revisione	MDL, AR, CS
22/01/2026	1.0	Sistemata CR01 per l'aggiunta	AR
22/01/2026	1.1	Prima stesura CR02	AR
24/01/2026	1.2	CR02 + Spiegazione dei trade off	AR
27/01/2026	1.3	Quinta revisione	MDL
28/01/2026	1.4	CR00	AR
15/02/2026	1.5	Sesta revisione	MDL



Sommario

Gestione Gruppo.....	2
Version.....	3
1 Introduzione.....	5
2 Change Request.....	7
2.1 Change Request CR00	7
2.2 Change Request CR01	8
2.1.1 Requisiti Funzionali	12
2.1.2 Requisiti Non Funzionali	12
2.3 Change Request CR02	13
3 Note	16



1 Introduzione

CodeSmile nasce come strumento per l'individuazione di *machine learning-specific code smell* in progetti Python, con l'obiettivo di supportare l'analisi della qualità del codice ML attraverso approcci di static analysis e tecniche AI-based. L'uso tipico consiste nel selezionare un input, un output e avviare l'analisi (personalizzata) tramite la GUI, la CLI o la WebApp.

Nel corso dello sviluppo del progetto e dell'analisi dell'architettura esistente, è emersa l'opportunità di **raffinare l'integrazione dei Large Language Models (LLM)**, per supportare la detection di smell a partire da un **prompt definito dall'utente**.

Parallelamente, nel corso dello sviluppo del progetto e delle attività di verifica, in particolare durante i test di sistema, sono emerse alcune **incoerenze comportamentali nella modalità GUI** rispetto alle specifiche definite.

Il seguente documento presenta un insieme di **change request correttive e additive**, mirate a raffinare l'integrazione dei Large Language Models (LLM) nella detection di ML-specific code smell, preservando integralmente la pipeline standard esistente e migliorare l'affidabilità della GUI.

In particolare, le change request introducono:

- **CR00:** correzione delle incoerenze di validazione e gestione delle opzioni operative nella GUI, emerse durante i test di sistema, al fine di garantire la conformità alle specifiche funzionali e un comportamento coerente e prevedibile per l'utente.
- **CR01 (GUI mode):** estensione della GUI per consentire all'utente di **abilitare/disabilitare l'analisi LLM, selezionare uno o più smell** su cui applicarla e **gestire prompt e catalogo degli smell** tramite interfacce dedicate (Prompt Engineering e Gestione Code Smell), supportate da una **struttura dati persistente** per smell/prompt/provider.
- **CR02 (CLI mode):** estensione della modalità a linea di comando per permettere l'uso della detection LLM anche via CLI, tramite **nuovi parametri opzionali** (abilitazione LLM, scelta tra LLM locale o provider API) e l'esecuzione su tutti gli smell per cui esiste un prompt disponibile, mantenendo la stessa logica LLM condivisa introdotta da CR01.

Nel complesso, le change request **non alterano il comportamento della pipeline esistente**: l'analisi LLM resta un comportamento aggiuntivo e attivabile a scelta dell'utente. L'impatto complessivo è pertanto limitato a:

- (i) controlli di configurazione e attivazione dell'analisi LLM (GUI e, con CR02, anche CLI);
- (ii) interfacce di gestione di prompt e smell (CR01);



(iii) persistenza dei dati e componente di orchestrazione LLM riutilizzabile tra le diverse modalità di utilizzo.

Nel contesto del progetto corrente, l'implementazione si concentra su **CR00 e CR01**, mentre **CR02** viene proposta come **estensione futura**, ponendo le basi per una completa integrazione dell'analisi LLM anche in modalità CLI.



2 Change Request

2.1 Change Request CR00

Change Request ID:	CR00		
Titolo CR:	Correzione delle incoerenze di validazione e gestione delle opzioni operative nella GUI rispetto alle specifiche di sistema		
Data di Identificazione:	28/01/2026	Priorità:	High
Richiesta da:	Martina De Lucia Alessandra Raia Carla Stefanile		
Tipo di CR			
La presente Change Request è classificabile come correttiva , in quanto nasce dall'individuazione di comportamenti errati e non conformi alle specifiche di sistema emersi durante l'esecuzione dei test di sistema sulla GUI. In particolare, la CR non introduce nuove funzionalità né modifica requisiti esistenti, ma mira a: • correggere mancate validazioni degli input; • allineare il comportamento della GUI alle regole operative già definite nelle specifiche; • risolvere discrepanze tra combinazioni di opzioni consentite e non consentite.			
Motivazione della richiesta			
Durante l'attività di System Testing , eseguita tramite l'esecuzione della suite test/gui_system_spec, sono stati individuati numerosi casi di test marcati come XFAIL , a indicare che il comportamento attuale della GUI non soddisfa i requisiti specificati. In particolare, su 43 test di sistema: • 18 risultano PASS; • 25 risultano XFAIL, tutti riconducibili a: ◦ assenza di controlli di validazione sugli input (es. max_workers); ◦ gestione incoerente delle modalità Multiple, Parallel e Resume; ◦ mancata segnalazione all'utente di opzioni ignorate o non applicabili. Questi comportamenti possono generare: • configurazioni di esecuzione semanticamente non valide; • risultati inattesi o fuorvianti per l'utente; • una divergenza tra GUI e comportamento atteso della modalità CLI.			
Obiettivo della change request			
L'obiettivo della CR00 è ripristinare la conformità della GUI alle specifiche di sistema , garantendo che: • tutte le opzioni configurabili siano correttamente validate; • le combinazioni non ammesse siano bloccate o gestite esplicitamente; • l'utente venga sempre informato quando un'opzione viene ignorata o disabilitata; • il comportamento della GUI sia coerente, prevedibile e allineato ai test di sistema definiti.			
Descrizione della soluzione proposta			
La soluzione proposta prevede l'introduzione di una logica di validazione centralizzata nella GUI, in grado di:			



1. Validare i parametri numerici

- Verifica che max_workers sia numerico e maggiore di zero.
- Blocco dell'avvio dell'analisi in caso di input non valido.

2. Gestire correttamente le combinazioni di modalità

- Spiegare l'impossibilità dell'uso della modalità Multiple quando è selezionato un solo progetto.
- Segnalare le combinazioni:
 - Multiple + Parallel con un solo progetto;
 - Multiple + Resume con un solo progetto;
 - Multiple + Parallel + Resume con un solo progetto.

3. Gestire le opzioni ignorate

- Visualizzare messaggi espliciti quando:
 - Resume viene ignorato in modalità Parallel;
 - Parallel o Resume vengono selezionati senza Multiple.

4. Allineare i messaggi informativi alle specifiche

- Stampare messaggi coerenti con quelli attesi nei test di sistema (es. "ignored without Multiple").

Impatto sull'architettura esistente

L'impatto sull'architettura esistente è limitato e localizzato, in quanto:

- non vengono modificati i servizi di analisi;
- non vengono introdotti nuovi moduli;
- non vengono alterate le interfacce pubbliche esistenti.

Le modifiche riguardano esclusivamente:

- la logica di validazione lato GUI;
- la gestione delle opzioni prima dell'invocazione dell'analisi.

Di conseguenza, il rischio di regressioni sul core del sistema è contenuto.

Punti critici

- Necessità di mantenere coerenza.
- Rischio di introdurre messaggi ridondanti o poco chiari se la validazione non è ben progettata.
- Importanza di aggiornare la GUI senza alterare il flusso di esecuzione esistente.

2.2 Change Request CR01

Change Request ID:	CR01		
Titolo CR:	Prompt-Driven ML-Specific Code Smell Detection + Prompt Engineering – <i>An Additive LLM-Based Extension for CodeSmile GUI mode</i>		
Data di Identificazione:	15/01/2026	Priorità:	High
Richiesta da:	Martina De Lucia Alessandra Raia Carla Stefanile		
Tipo di CR			
La presente change request è classificabile come additiva (in relazione allo standard ISO/IEC/IEEE 14764), in quanto introduce una nuova funzionalità opzionale che			



estende le capacità di CodeSmile senza modificare o correggere il comportamento esistente.

Non si tratta di una modifica correttiva, adattiva, perfettiva o preventiva, poiché:

- non risolve difetti esistenti,
- non risponde a cambiamenti imposti dall'ambiente,
- non nasce come intervento di riduzione del rischio, ma come estensione funzionale controllata.

Motivazione della richiesta

L'attuale implementazione di CodeSmile offre due modalità di rilevamento:

1. Static Analysis Tool: rileva code smell attraverso regole hardcoded basate su AST parsing
2. AI-Based Tool: utilizza LLM con modelli pre-addestrati (fine-tuned) su dataset specifici

Entrambi gli approcci hanno limitazioni:

- Lo Static Analysis è rigido e non può adattarsi a nuove categorie di smell senza intervento dello sviluppatore
- L'AI-Based Tool richiede un'ingegneria complessa di dataset di fine-tuning e non consente agli utenti di personalizzare il rilevamento

L'attuale modalità di utilizzo di CodeSmile è centrata su un flusso di analisi "standard" che non prevede:

- selezione mirata degli smell da valutare tramite LLM;
- definizione di prompt dedicati per smell e loro raffinamento nel tempo;
- gestione controllata di prompt predefiniti, modifiche temporanee e aggiornamenti permanenti;
- aggiunta di smell ulteriori non presenti nel catalogo iniziale, descritti tramite prompt, con possibilità di validazione rapida su un progetto;
- eliminazione di code smell il cui prompt engineering è risultato fallimentare.

Si potrebbe ambire a uno scenario in cui l'utente sceglie gli smell di interesse e scrive un prompt dedicato per ciascuno smell; il tool utilizza tali prompt per eseguire la detection tramite LLM.

Per rendere questa modalità utilizzabile e verificabile, è necessario introdurre un'interfaccia e una persistenza strutturata dei prompt e della lista degli smell.

Obiettivo della change request

L'obiettivo della CR01 è introdurre tre capacità complementari:

A) Abilitazione e configurazione della detection LLM dalla GUI principale

Consentire all'utente, nella schermata di avvio dell'analisi, di:

- abilitare/disabilitare la modalità LLM;
- scegliere il tipo di esecuzione (LLM locale vs chiamata API a provider proprietari);
- selezionare uno o più smell su cui applicare la detection LLM.

B) Introduzione di una nuova interfaccia "Prompt Engineering"

Consentire all'utente di:

- gestire la lista degli smell disponibili in modalità LLM;
- associare e modificare un prompt per ciascuno smell (prompt engineering);
- salvare le modifiche come temporanee (draft) e/o permanenti (aggiornamento del prompt predefinito);



- eseguire un test su folder utilizzando il prompt corrente e analizzare i risultati.

C) Introduzione di una nuova interfaccia “Gestione Code Smell”

Consentire all’utente di:

- aggiungere, eliminare o modificare uno smell.

L’estensione deve mantenere invariato il flusso standard di CodeSmile: la modalità LLM è opzionale e non deve impedire l’uso del tool in modalità tradizionale.

Descrizione della soluzione proposta

1) Estensione della GUI “Run”

La finestra principale viene estesa con l’introduzione di un pulsante “LLM” che consenta:

- attivazione/disattivazione della modalità LLM;
- selezione provider LLM:
 - Locale: selezione da elenco di modelli locali disponibili;
 - API: selezione del provider proprietario (es. OpenAI/Claude/Gemini);
- selezione di uno o più smell da rilevare in modalità LLM (multi-selezione).

La pipeline standard rimane disponibile senza modifiche: l’analisi LLM è un comportamento aggiuntivo attivabile a scelta dell’utente.

2) Nuova interfaccia “Prompt Engineering”

Si introduce una finestra dedicata alla gestione di smell e prompt, con le seguenti funzionalità:

- Selezione smell e visualizzazione prompt
 - selezione di uno smell → visualizzazione del prompt temporaneo associato;
 - possibilità di visualizzare il prompt di default o di visualizzare il temporaneo;
 - possibilità di modifica del prompt temporaneo nella finestra di editing.
- Gestione versioni (temporaneo vs default)
 - salvataggio temporaneo (draft) per sperimentazione e validazione;
 - salvataggio permanente che aggiorna il prompt di default associato allo smell usato nella fase di detection.
- Test del prompt
 - selezione input folder e output folder;
 - esecuzione di un test che usa il prompt attualmente visualizzato (temporaneo o default);
 - consultazione sintetica dei risultati prodotti (e accesso al report).

3) Nuova interfaccia “Gestione Code Smell”

- Gestione smell
 - aggiunta di un nuovo smell con descrizione;
 - eventuale rimozione/modifica di smell creati dall’utente.

4) Persistenza e struttura dati

Per rendere ripetibile e verificabile la nuova modalità, è necessario introdurre una persistenza strutturata che contenga:



- catalogo degli smell disponibili in modalità LLM;
- prompt di default per smell;
- prompt temporanei e relative modifiche;
- elenco dei provider LLM disponibili (locali) e relative configurazioni

Supporto a provider diversi (vincolo architetturale)

La CR richiede che l'utente possa scegliere tra:

- **esecuzione tramite LLM locale,**
- **esecuzione tramite provider API.**

Per evitare accoppiamento e facilitare manutenzione e test, la soluzione prevede un'astrazione che separi la logica di orchestrazione della detection LLM dall'implementazione concreta del provider. È consigliata l'adozione del **Bridge Pattern** come riferimento progettuale.

Impatto sull'architettura esistente

La modifica proposta:

- non introduce breaking changes;
- non modifica la pipeline standard (analisi statica e reportistica esistente);
- aggiunge:
 - controlli GUI per attivare/configurare LLM e selezionare smell;
 - una seconda GUI "Prompt Engineering";
 - una terza GUI "Gestione Code Smell"
 - una persistenza per catalogo smell/prompt/provider;
 - una componente di orchestrazione della detection LLM e integrazione dei risultati nei report.

Il sistema resta pienamente utilizzabile anche senza attivare la modalità LLM.

Punti critici

I seguenti punti critici non rappresentano blocker per l'adozione della change request, ma aspetti noti e gestibili che emergono dall'introduzione di una modalità LLM-based.

1. Normalizzazione dei risultati

L'output dell'LLM deve essere convertibile in risultati coerenti e tracciabili. È necessario definire uno schema di output atteso (preferibilmente strutturato) per ridurre ambiguità.

2. Gestione dei prompt (temporanei vs default)

Occorre distinguere chiaramente tra prompt temporanei (sperimentazione) e prompt di default (baseline), evitando che modifiche non validate compromettano i default.

3. Latenza e costi (provider API)

L'uso di provider API può introdurre costi e latenze. La selezione mirata degli smell e la funzione di test aiutano a contenere e valutare l'impatto.

4. Coerenza del catalogo smell

L'aggiunta di smell custom richiede regole per ID univoci e gestione consistente tra Prompt Engineering, GUI Run e report.



2.1.1 Requisiti Funzionali

I requisiti funzionali descrivono in modo puntuale i comportamenti e le funzionalità che il sistema deve offrire per supportare la change request proposta.

Nel contesto di CR01, tali requisiti definiscono le modalità con cui l'utente può abilitare, configurare e utilizzare la detection basata su Large Language Model (LLM), nonché interagire con i prompt e con il catalogo degli smell.

L'obiettivo dei requisiti funzionali è formalizzare le nuove capacità introdotte dall'estensione LLM-based di CodeSmile, garantendo che esse siano **opzionali, controllabili dall'utente e integrate senza alterare il flusso standard esistente**.

I requisiti elencati coprono sia l'interazione dalla GUI principale (modalità "Run"), sia le funzionalità offerte dalle nuove interfacce, assicurando coerenza tra configurazione, persistenza e utilizzo operativo della detection LLM.

- **FR1 — Modalità LLM opzionale:** l'utente può abilitare/disabilitare LLM-mode dalla GUI principale.
- **FR2 — Selezione provider:** l'utente può scegliere provider locale o API e selezionare un elemento da elenco.
- **FR3 — Selezione smells:** l'utente può selezionare uno o più smell su cui eseguire la detection LLM.
- **FR4 — Prompt per smell:** per ogni smell è associato un prompt; l'utente può visualizzarlo e modificarlo nell'interfaccia di Prompt Engineering.
- **FR5 — Salvataggio draft:** l'utente può salvare un prompt modificato come temporaneo (draft) senza alterare i default.
- **FR6 — Salvataggio permanente:** l'utente può salvare il prompt come default per quello smell (usato nella fase di detection).
- **FR7 — Gestione smell custom:** l'utente può aggiungere un nuovo smell (nome + descrizione) e renderlo selezionabile in LLM-mode, può rimuovere uno smell esistente o modificarne la descrizione facoltativa.
- **FR8 — Test su folder:** dall'interfaccia di Prompt Engineering l'utente può eseguire un test su una cartella e generare risultati consultabili.

2.1.2 Requisiti Non Funzionali

I requisiti non funzionali descrivono le proprietà di qualità che il sistema deve rispettare a seguito dell'introduzione della nuova funzionalità LLM-based.

Essi non riguardano *il cosa* il sistema fa, ma *il come* tali funzionalità devono essere implementate e supportate, in termini di affidabilità, manutenibilità, estendibilità e usabilità.

Nel caso della presente change request, i requisiti non funzionali assumono un ruolo particolarmente rilevante, poiché l'integrazione di LLM introduce nuove criticità legate



a dipendenze esterne, costi, latenza e variabilità dei risultati.

I requisiti definiti mirano pertanto a garantire che l'estensione proposta sia **robusta, testabile, facilmente estendibile e trasparente per l'utente**, preservando al contempo la stabilità e la qualità complessiva di CodeSmile.

- **NFR1 — Testabilità:** disponibilità di un provider “mock” per test automatici senza chiamate esterne.
- **NFR2 — Estendibilità:** aggiunta di nuovi provider e nuovi smell senza modifiche invasive.
- **NFR3 — Robustezza:** gestione degli errori dell'utente.
- **NFR4 — Usabilità:** separazione tra schermata “Run”, “Prompt Engineering” e “Gestione Code Smell” per evitare overload dell'utente.
- **NFR5 — Tracciabilità:** risultati LLM distinguibili nel report.

2.3 Change Request CR02

Change Request ID:	CR02		
Titolo CR:	Prompt-Driven ML-Specific Code Smell Detection – An Additive LLM-Based Extension for CodeSmile CLI mode		
Data di Identificazione:	22/01/2026	Priorità:	Low
Richiesta da:	Martina De Lucia Alessandra Raia Carla Stefanile		
Tipo di CR			
La presente change request è classificabile come additiva (in relazione allo standard ISO/IEC/IEEE 14764), in quanto introduce una nuova funzionalità opzionale che estende le capacità di CodeSmile senza modificare o correggere il comportamento esistente. Non si tratta di una modifica correttiva, adattiva, perfettiva o preventiva, poiché: • non risolve difetti esistenti, • non risponde a cambiamenti imposti dall’ambiente, • non nasce come intervento di riduzione del rischio, ma come estensione funzionale controllata.			
Motivazione della richiesta			
Con l’implementazione della CR01, e cioè con l’utilizzo di LLM, la possibilità di effettuare Prompt Engineering e gestire i code smell, CodeSmile ha migliorato le prestazioni, con il fine di poter essere un ottimo strumento per gli sviluppatori che lavorano e si interessano ai code smell. Le funzionalità aggiunte sono, però, fruibili solo da GUI. Per gli sviluppatori che intendono usare lo strumento da linea di comando (CLI), l’attuale modalità di utilizzo di CodeSmile è centrata su un flusso di analisi “standard” che non prevede l’utilizzo degli LLM. Si potrebbe ambire a uno scenario in cui l’utente sceglie di eseguire la detection tramite LLM di tutti gli smell collezionati e per cui esiste un prompt lanciabile tramite LLM.			



Per rendere questa modalità utilizzabile e verificabile, è necessario introdurre un'espansione al comando di avvio della detection da linea di comando.

Obiettivo della change request

L'obiettivo della CR01 è:

A) Abilitazione della detection LLM dalla CLI

Consentire all'utente di:

- abilitare la modalità LLM da linea di comando;
- scegliere il tipo di esecuzione (LLM locale vs chiamata API a provider proprietari);

L'estensione deve mantenere invariato il flusso standard di CodeSmile: la modalità LLM è opzionale e non deve impedire l'uso del tool in modalità tradizionale.

Descrizione della soluzione proposta

1) Estensione della CLI "Run"

La modalità CLI viene estesa introducendo nuovi parametri opzionali che consentono di attivare la detection basata su LLM anche da linea di comando.

In particolare, il comando di avvio dell'analisi viene arricchito con opzioni che permettono di:

- abilitare o disabilitare esplicitamente la modalità LLM;
- selezionare la tipologia di esecuzione LLM (modello locale oppure provider API);
- eseguire la detection LLM su tutti gli ML-specific code smell per cui è disponibile un prompt associato.

In assenza dei nuovi parametri, il comportamento della CLI rimane invariato e continua a eseguire esclusivamente la pipeline standard di CodeSmile, garantendo piena retrocompatibilità.

Impatto sull'architettura esistente

La change request CR02 non introduce modifiche strutturali invasive all'architettura di CodeSmile.

In particolare:

- non altera la pipeline di analisi standard già esistente;
- non modifica le funzionalità GUI introdotte con la CR01;
- non introduce dipendenze obbligatorie verso i modelli LLM.

L'impatto architetturale è limitato a:

- un'estensione del layer CLI per la gestione di nuovi flag e parametri;
- il riutilizzo della componente di orchestrazione LLM già presente;
- l'integrazione dei risultati LLM nei report generati dalla modalità CLI, mantenendo la distinzione tra output standard e output LLM-based.

La soluzione proposta favorisce la coesione architetturale, garantendo che la logica LLM sia centralizzata e condivisa tra GUI e CLI, migliorando manutenibilità ed estendibilità del sistema.

Punti critici

L'introduzione della detection LLM in modalità CLI presenta alcuni aspetti critici, che non costituiscono blocker ma richiedono attenzione progettuale:

1. Gestione dell'usabilità da linea di comando

L'introduzione di nuove opzioni CLI deve mantenere un buon equilibrio tra flessibilità e semplicità, evitando comandi eccessivamente complessi o difficili da comprendere per l'utente.



2. Tracciabilità e chiarezza dell'output

I risultati prodotti dalla detection LLM devono essere chiaramente distinguibili da quelli della pipeline standard anche in modalità CLI, sia a console sia nei file di report generati.

3. Allineamento con la CR01

La CR02 dipende concettualmente dalle funzionalità introdotte con la CR01; eventuali evoluzioni future della gestione dei prompt o dei provider devono preservare la compatibilità con l'utilizzo da linea di comando.



3 Note

Trade-off tempo-modifiche:

nel contesto del progetto attuale, l'implementazione si è concentrata esclusivamente sulle Change Request **CR00 e CR01**.

La scelta di includere **CR00** è stata determinata dall'esito dei test di sistema sulla modalità GUI, che hanno evidenziato comportamenti non conformi alle specifiche. La CR00 risulta quindi necessaria per garantire **affidabilità, prevedibilità e conformità** del sistema prima dell'introduzione di estensioni funzionali.

Parallelamente, l'implementazione di **CR01** richiede un impegno significativo per l'integrazione dei *Large Language Models*, la riorganizzazione delle componenti coinvolte e l'introduzione di meccanismi di persistenza e orchestrazione riutilizzabili, mantenendo invariata la pipeline standard esistente.

La Change Request **CR02** non è pertanto oggetto di implementazione nella versione corrente di CodeSmile, ma viene proposta come **estensione concettuale e progettuale**, finalizzata a delineare possibili evoluzioni future del tool. Tale scelta è stata guidata dalla necessità di assicurare **stabilità, coerenza e verificabilità** delle funzionalità LLM-based entro i vincoli temporali del progetto.

In particolare, CR02 evidenzia come le funzionalità introdotte con CR01 costituiscano una base modulare e riutilizzabile per estendere la detection LLM anche in modalità CLI, contribuendo a una copertura più completa dei casi d'uso e a una maggiore flessibilità per gli sviluppatori.

Use Case Document:

al fine di non appesantire il presente documento con ulteriori dettagli di tipo comportamentale, la modellazione dei casi d'uso relativi alle funzionalità introdotte dalla change request CR01 è stata raccolta in un documento separato: UseCase.

In tale documento vengono illustrati il modello dei casi d'uso e i principali scenari di interazione tra l'utente e il sistema, con riferimento alle nuove funzionalità LLM-based e alla loro integrazione nel flusso operativo di CodeSmile.

Il documento UseCase fornisce quindi una vista complementare ai requisiti funzionali qui definiti, supportando la comprensione del comportamento del sistema dal punto di vista dell'utente.